

Object-Oriented Programming in Python

**Prepared by
Asif Farooq**

Inheritance and its Types

- ▶ Inheritance is a mechanism that allows a class to acquire the properties of another class.
- ▶ Parent class (Base class) and Child class (Derived class).
- ▶ Promotes code reusability.

Types of Inheritance

- ▶ Single Inheritance: Inherits from Single Class
- ▶ Multiple Inheritance: Inherits from Multiple Classes
- ▶ Multilevel inheritance is a type of inheritance that involves a class inheriting from another derived class

The super() Function

- ▶ `super()` is used to call a method from the parent class.
- ▶ Useful when a child class overrides a method but still wants to call the parent's method.
- ▶ Example:

```
class Parent:
    def show(self):
        print('Parent Method')

class Child(Parent):
    def show(self):
        super().show()
        print('Child Method')

obj = Child()
obj.show()
```

Class Methods

- ▶ A class method works at the class level, not the instance level.
- ▶ Defined using `@classmethod` and takes `cls` as the first parameter.
- ▶ Can modify class attributes and is shared across instances.
- ▶ Example:

```
class Employee:
```

```
    company = 'TechCorp'
```

```
    @classmethod
```

```
    def set_company(cls, new_name):
```

```
        cls.company = new_name
```

```
print(Employee.company) # Output: TechCorp
```

```
Employee.set_company('NewTech')
```

```
print(Employee.company) # Output: NewTech
```

Getters & Setters in Python

- ▶ Used to control access to class attributes.
- ▶ `@property` is used for getters, and `@<property\_name>.setter` for setters.
- ▶ Example:

```
class Student:
```

```
    def __init__(self, age):
```

```
        self._age = age
```

```
    @property
```

```
    def age(self):
```

```
        return self._age
```

```
    @age.setter
```

```
    def age(self, value):
```

```
        if value < 0:
```

```
            raise ValueError('Age cannot be negative')
```

```
        self._age = value
```

```
s = Student(20)
```

```
s.age = 25 # Calls setter
```

```
print(s.age) # Calls getter
```

Operator Overloading

- ▶ A programming technique that allows programmers to redefine the behavior of operators for user-defined data types.
- ▶ Allows custom behavior for operators like ``+``, ``-``, ``*``, etc.
- ▶ Implemented using special methods like ``__add__``, ``__sub__``, etc.
- ▶ `print(10 + 5)` # 15 (int addition)
- ▶ `print("Hello " + "World")` # Hello World (string concat)
- ▶ `print([1, 2] + [3, 4])` # [1, 2, 3, 4] (list concat)

Programs

- ▶ Create a class (2-D vector) and use it to create another class representing a 3-D vector.
- ▶ Create a class 'Student' from a class 'School' and further create a class 'Subjects' from 'Student'. Add a method 'show_name()' to class 'Student'
- ▶ Implement getters and setters in your program for displaying the first_name_middle_name and last_name of a school